Connect to a Server with SSL: How to Use the Certificate

Explore how to securely connect Python REST clients to servers by loading and validating SSL certificates. Learn to configure the requests library with certificate paths for proper verification, ensuring safe API communication and avoiding SSL handshake errors.

How to use the certificate

Ignoring a self-signed cert is fine while developing and testing a REST client. However, for the release phase, the application needs to load and validate the certificate sent by the server using it.

The self-signed certs aren't available in the certs cache/pool of the operating system. Hence, for an application to use them, it needs to do the following:

1. Load the cert.
2. Pass the cert to the `requests` library so that it can use the cert for validation.

The `requests` library loads and passes the cert automatically via the `ssl_verify` option. It's the same `ssl_verify` option we used to ignore cert validation in the last lesson.

The service that we'll use for checking our code will be https://self-signed.badssl.com, as it provides a self-signed certificate for testing purpo
We'll also need to export the cert of the site to be used in the code.

Ask

> **Note:** We'll reuse and update the code that we've developed so far.

Let's see how to do that.

Steps

1. Open `client.py`.
2. Add a new parameter to the constructor named `cert_path`. Set its default value to `None`.

```
def __init__(self, url: str, ssl_verify=True, cert_path=None):
```

3. Remove the statement that sets the value of `ssl_verify` within the constructor.
4. Add an `if-else` block within the constructor to check whether the `cert_path` is `None`.

```
if cert_path:
else:
```

5. Assign the `cert_path` parameter to the `ssl_verify` instance variable within the `if` block.

```
if cert_path:self.ssl_verify = cert_path
```

6. Assign the `ssl_verify` parameter to the `ssl_verify` instance variable in the `else` block.

```
else:self.ssl_verify = ssl_verify
```

7. Update the first print statement In the main block to indicate that the cert isn't being passed.

Ask

```
print("Client with Verification Enabled and not passing cert")
```

8. Update the second print statement to indicate that the cert is being passed.

```
print("Client with Verification Enabled and passing cert")
```

9. Remove the existing `client_no_verify` instance and create a new instance of `Client` by passing the path to the cert via the `cert_path` argument. Assign the new instance to `client_verify_cert` variable.

```
client_verify_cert = Client('https://self-signed.badssl.com', cert_path=str(
```

10. Update the print statement that displays the content to use `client_verify_cert`.

```
print(client_verify_cert.get())
```

11. Run the script. This will give the result of both calls, where the cert verification fails, as we didn't load and pass the self-signed certificate as indicated by the exception printed out, and the second, where the server's cert is successfully verified using the cert that we passed to the library and the response content is printed.

Show client.py output

How it works

Python `requests` library can accept the path to the CA cert and use it while validating the cert of the server. We make use of this functionality by passi path of the downloaded cert to the constructor of `client_verify_cert`.

Ask

In the constructor, the `if` condition checks whether the `cert_path` is null or not. If it's not null, the path is assigned to the `ssl_verify` instance variable, which, in turn, is passed to the `get` method of `requests` library. Due to this, when `get` is called on the `client_verify_cert` instance, the call succeeds.

The following diagram depicts the request/response cycle between the client and the server when client verification is enabled but a certificate is not passed:

Client with verification enabled and not passing certificate

The following diagram depicts the request-response cycle between the client and the server when client verification is enabled, and the correct certificate is passed:

Ask

Client with verification enabled and passing certificate

Ask

Try it yourself

- The widget below shows the code created during the last lesson.
- Follow the steps above to complete the code for this lesson.
- Click "Run" to execute the code and display the result.

> **Note:** The `badssl-com.pem` file is updated periodically. If the code throws an SSL handshake exception, the latest certificate must be updated from https://self-signed.badssl.com in the `badssl-com.pem` file.

Files

client.py

badssl-...

client.py

```
## This is code created during the last lesson.
## Modify the code according to the lesson steps.
#
#
from ssl import SSLError, SSLCertVerificationError

import requests


class Client(object):
    def __init__(self, url: str, ssl_verify=True):
        self.url = url
        self.ssl_verify = ssl_verify
```

The PEM file contains the server SSL certificate. When we run the code and pass the certificate, we get the output. However, an exception is raised when the certificate is not passed.

Solution

If stuck, refer to the final code below for help.

Ask

Please note that all the files below are read-only.

Files

client.py

badssl-...

client.py

```
from ssl import SSLError, SSLCertVerificationError

import requests


class Client(object):
```

Ask